Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy

En Li, Zhi Zhou, and Xu Chen
School of Data and Computer Science
Sun Yat-sen University, Guangzhou, China

Abstract

As the backbone technology of machine learning, deep neural networks (DNNs) have have quickly ascended to the spotlight. Running DNNs on resource-constrained mobile devices is, however, by no means trivial, since it incurs high performance and energy overhead. While offloading DNNs to the cloud for execution suffers unpredictable performance, due to the uncontrolled long wide-area network latency. To address these challenges, in this paper, we propose Edgent, a collaborative and on-demand DNN co-inference framework with device-edge synergy. Edgent pursues two design knobs: (1) DNN partitioning that adaptively partitions DNN computation between device and edge, in order to leverage hybrid computation resources in proximity for real-time DNN inference. (2) DNN right-sizing that accelerates DNN inference through early-exit at a proper intermediate DNN layer to further reduce the computation latency. The prototype implementation and extensive evaluations based on Raspberry Pi demonstrate Edgent's effectiveness in enabling on-demand low-latency edge intelligence.

I. Introduction & Related Work

As the backbone technology supporting modern intelligent mobile applications, Deep Neural Networks (DNNs) represent the most commonly adopted machine learning technique and have become increasingly popular. Due to DNNs's ability to perform highly accurate and reliable inference tasks, they have witnessed successful applications in a broad spectrum of domains from computer vision [1] to speech recognition [2] and natural language processing [3]. However, as DNN-based applications typically require tremendous amount of computation, they cannot be well supported by today's mobile devices with reasonable latency and energy consumption.

In response to the excessive resource demand of DNNs, the traditional wisdom resorts to the powerful cloud datacenter for training and evaluating DNNs. Input data generated from mobile devices is sent to the cloud for processing, and then results are sent back to the mobile devices after the inference. However, with such a cloud-centric approach, large amounts of data (e.g., images and videos) are uploaded to the remote cloud via a long wide-area network data transmission, resulting in high end-to-end latency and energy consumption of the mobile devices. To alleviate the latency and energy bottlenecks of cloud-centric approach, a better solution is to exploiting the emerging edge computing paradigm. Specifically, by pushing the cloud capabilities from the network core to the network edges (e.g., base stations and WiFi access points) in close proximity to devices, edge computing enables low-latency and energy-efficient DNN inference.

While recognizing the benefits of edge-based DNN inference, our empirical study reveals that the performance of edge-based DNN inference is highly sensitive to the available bandwidth between the edge server and the mobile device. Specifically, as the bandwidth drops from 1Mbps to 50Kbps, the latency of edge-based DNN inference climbs from 0.123s to 2.317s and becomes on par with the latency of local processing on the device. Then, considering the vulnerable and volatile network bandwidth in realistic environments (e.g., due to user mobility and bandwidth contention among various Apps), a natural

En Li, Zhi Zhou, and Xu Chen, "Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy," The SIGCOMM Workshop on Mobile Edge Communications, Budapest, Hungary, August 21-23, 2018.

The corresponding author is Xu Chen.

边缘智能：按需深度学习模型与设备边缘协同的协同推理

En Li、Zhi Zhou 和 Xu Chen 中山
大学数据与计算机科学学院，中国广州

抽象的

作为机器学习的支柱技术，深度神经网络（DNN）迅速成为人们关注的焦点。然而，在资源受限的移动设备上运行 DNN 绝非易事，因为它会带来高性能和能源开销。将 DNN 卸载到云端执行时，由于不受控制的长广域网延迟，性能会受到不可预测的影响。为了应对这些挑战，在本文中，我们提出了 Edgent，一种具有设备边缘协同作用的协作式按需 DNN 协同推理框架。 Edgent 追求两个设计要点：(1) DNN 分区，在设备和边缘之间自适应地分区 DNN 计算，以便利用邻近的混合计算资源进行实时 DNN 推理。 (2) DNN 大小调整，通过在适当的中间 DNN 层提前退出来加速 DNN 推理，以进一步减少计算延迟。基于 Raspberry Pi 的原型实现和广泛评估证明了 Edgent 在实现按需低延迟边缘智能方面的有效性。

一、引言及相关工作

作为支持现代智能移动应用的骨干技术，深度神经网络（DNN）代表了最常用的机器学习技术，并且越来越受欢迎。由于 DNN 能够执行高度准确和可靠的推理任务，因此它们在从计算机视觉 [1] 到语音识别 [2] 和自然语言处理 [3] 的广泛领域中得到了成功的应用。然而，由于基于 DNN 的应用程序通常需要大量计算，因此当今的移动设备无法以合理的延迟和能耗很好地支持它们。

为了应对 DNN 过多的资源需求，传统智慧求助于强大的云数据中心来训练和评估 DNN。移动设备生成的输入数据被发送到云端进行处理，然后推理后将结果发送回移动设备。然而，采用这种以云为中心的方法，大量数据（例如图像和视频）通过长广域网数据传输上传到远程云，导致移动设备的端到端延迟和能耗较高。为了缓解以云为中心的方法的延迟和能源瓶颈，更好的解决方案是利用新兴的边缘计算范例。具体来说，通过将云功能从网络核心推送到靠近设备的网络边缘（例如基站和 WiFi 接入点），边缘计算可实现低延迟且节能的 DNN 推理。

在认识到基于边缘的 DNN 推理的好处的同时，我们的实证研究表明，基于边缘的 DNN 推理的性能对边缘服务器和移动设备之间的可用带宽高度敏感。具体来说，随着带宽从 1Mbps 下降到 50 Kbps，基于边缘的 DNN 推理的延迟从 0.123 秒攀升至 2.317 秒，与设备上本地处理的延迟持平。然后，考虑到现实环境中脆弱且不稳定的网络带宽（例如，由于用户移动性和各种应用程序之间的带宽争用），自然

En Li, Zhi Zhou, and Xu Chen, “Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy,” The SIGCOMM Workshop on Mobile Edge Communications, Budapest, Hungary, August 21-23, 2018.

The corresponding author is Xu Chen.

question is that can we further improve the performance (i.e., latency) of edge-based DNN execution, especially for some mission-critical applications such as VR/AR games and robotics [4].

To answer the above question in the positive, in this paper we proposed Edgent, a deep learning model co-inference framework with device-edge synergy. Towards low-latency edge intelligence[1], Edgent pursues two design knobs. The first is DNN partitioning, which adaptively partitions DNN computation between mobile devices and the edge server based on the available bandwidth, and thus to take advantage of the processing power of the edge server while reducing data transfer delay. However, worth noting is that the latency after DNN partition is still restrained by the rest part running on the device side. Therefore, Edgent further combines DNN partition with DNN right-sizing which accelerates DNN inference through early-exit at an intermediate DNN layer. Needless to say, early-exit naturally gives rise to the latency-accuracy tradeoff (i.e., early-exit harms the accuracy of the inference). To address this challenge, Edgent jointly optimizes the DNN partitioning and right-sizing in an on-demand manner. That is, for mission-critical applications that typically have a pre-defined deadline, Edgent maximizes the accuracy without violating the deadline. The prototype implementation and extensive evaluations based on Raspberry Pi demonstrate Edgent's effectiveness in enabling on-demand low-latency edge intelligence.

While the topic of edge intelligence has began to garner much attention recently, our study is different from and complementary to existing pilot efforts. On one hand, for fast and low power DNN inference at the mobile device side, various approaches as exemplified by DNN compression and DNN architecture optimization has been proposed [5], [6], [7], [8], [9]. Different from these works, we take a scale-out approach to unleash the benefits of collaborative edge intelligence between the edge and mobile devices, and thus to mitigate the performance and energy bottlenecks of the end devices. On the other hand, though the idea of DNN partition among cloud and end device is not new [10], realistic measurements show that the DNN partition is not enough to satisfy the stringent timeliness requirements of mission-critical applications. Therefore, we further apply the approach of DNN right-sizing to speed up DNN inference.

II. Background & Motivation

In this section, we first give a primer on DNN, then analyse the inefficiency of edge- or device-based DNN execution, and finally illustrate the benefits of DNN partitioning and right-sizing with device-edge synergy towards low-latency edge intelligence.

A. A Primer on DNN

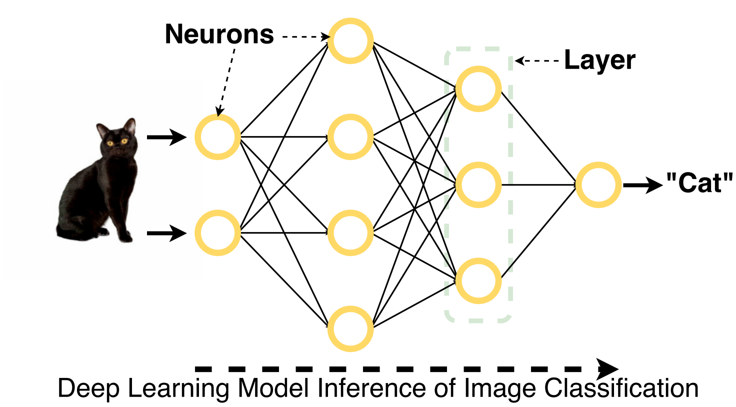

Figure 1. A 4-layer DNN for computer vision

[1]As an initial investigation, in this paper we focus on the execution latency issue. We will also consider the energy efficiency issue in a future study.

阙 问题是我们能否进一步提高基于边缘的 DNN 执行的性能（即延迟），
埃斯特别是对于一些关键任务应用，例如 VR/AR 游戏和机器人 [4]。

为了正面回答上述问题，本文提出了 Edgent，一种具有设备边缘协同作用的深度学习模型协同推理框架。为了实现低延迟边缘智能[1]，Edgent 追求两个设计旋钮。第一个是 DNN 分区，它根据可用带宽在移动设备和边缘服务器之间自适应地分区 DNN 计算，从而利用边缘服务器的处理能力，同时减少数据传输延迟。但值得注意的是，DNN 分区后的延迟仍然受到设备端运行的其余部分的限制。因此，Edgent 进一步将 DNN 分区与 DNN 大小调整相结合，通过中间 DNN 层的提前退出来加速 DNN 推理。不用说，提前退出自然会引起延迟与准确性的权衡（即提前退出会损害推理的准确性）。为了应对这一挑战，Edgent 以按需方式联合优化 DNN 分区和调整大小。也就是说，对于通常有预先定义的截止日期的关键任务应用程序，Edgent 可以在不违反截止日期的情况下最大限度地提高准确性。基于 Raspberry Pi 的原型实现和广泛评估证明了 Edgent 在实现按需低延迟边缘智能方面的有效性。

虽然边缘智能的话题最近开始引起人们的广泛关注，但我们的研究与现有的试点工作不同，但又是互补的。一方面，为了在移动设备端实现快速、低功耗的 DNN 推理，人们提出了以 DNN 压缩和 DNN 架构优化为代表的各种方法[5]、[6]、[7]、[8]、[9]。与这些工作不同的是，我们采用横向扩展的方法来释放边缘和移动设备之间协作边缘智能的优势，从而减轻终端设备的性能和能源瓶颈。另一方面，尽管云和终端设备之间的 DNN 划分的想法并不新鲜[10]，但实际测量表明 DNN 划分不足以满足关键任务应用程序严格的时效性要求。因此，我们进一步应用 DNN 调整大小的方法来加速 DNN 推理。

二.背景与动机

在本节中，我们首先介绍 DNN，然后分析基于边缘或基于设备的 DNN 执行的低效率，最后说明 DNN 分区和通过设备边缘协同调整大小以实现低延迟边缘智能的好处。

A. A Primer on DNN

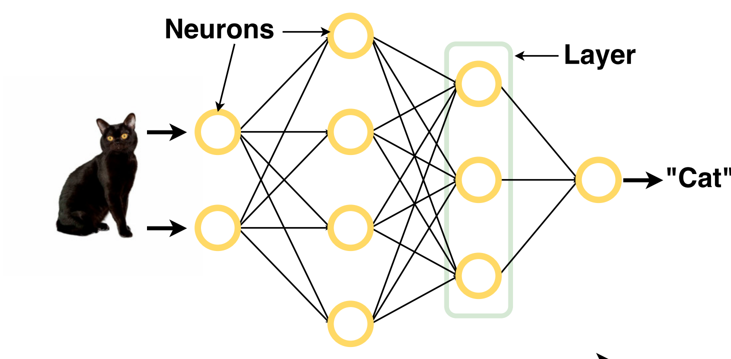

图 1. 用于计算机视觉的 4 层 DNN

1A这是初步调查，在本文中我们重点关注执行延迟问题。我们还将考虑能源效率问题
未来电子学习。

DNN represents the core machine learning technique for a broad spectrum of intelligent applications spanning computer vision, automatic speech recognition and natural language processing. As illustrated in Fig. 1, computer vision applications use DNNs to extract features from an input image and classify the image into one of the pre-defined categories. A typical DNN model is organized in a directed graph which includes a series of inner-connected layers, and within each layer comprising some number of nodes. Each node is a neuron that applies certain operations to its input and generates an output. The input layer of nodes is set by raw data while the output layer determines the category of the data. The process of passing forward from the input layer to the out layer is called model inference. For a typical DNN containing tens of layers and hundreds of nodes per layer, the number of parameters can easily reach the scale of millions. Thus, DNN inference is *computational intensive*. Note that in this paper we focus on DNN inference, since DNN training is generally delay tolerant and is typically conducted in an off-line manner using powerful cloud resources.

B. Inefficiency of Device- or Edge-based DNN Inference

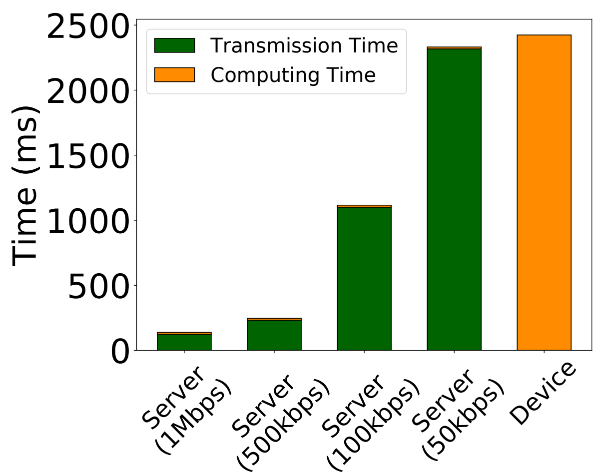

Figure 2. AlexNet Runtime

Currently, the status quo of mobile DNN inference is either direct execution on the mobile devices or offloading to the cloud/edge server for execution. Unfortunately, both approaches may suffer from poor performance (i.e., end-to-end latency), being hard to well satisfy real-time intelligent mobile applications (e.g., AR/VR mobile gaming and intelligent robots) [11]. As illustration, we take a Raspberry Pi tiny computer and a desktop PC to emulate the mobile device and edge server respectively, running the classical AlexNet [12] DNN model for image recognition over the Cifar-10 dataset [13]. Fig. 2 plots the breakdown of the end-to-end latency of different approaches under varying bandwidth between the edge and mobile device. It clearly shows that it takes more than 2s to execute the model on the resource-limited Raspberry Pi. Moreover, the performance of edge-based execution approach is dominated by the input data transmission time (the edge server computation time keeps at $\sim$10ms) and thus highly *sensitive* to the available bandwidth. Specifically, as the available bandwidth jumps from 1Mbps to 50Kbps, the end-to-end latency climbs from 0.123s to 2.317s. Considering the network bandwidth resource scarcity in practice (e.g., due to network resource contention among users and apps) and computing resource limitation on mobile devices, both of the device- and edge-based approaches are challenging to well support many emerging real-time intelligent mobile applications with stringent latency requirement.

DNN 代表了计算机视觉、自动语音识别和自然语言处理等广泛智能应用的核心机器学习技术。如图 1 所示，计算机视觉应用程序使用 DNN 从输入图像中提取特征，并将图像分类为预定义的类别之一。典型的 DNN 模型以有向图的形式组织，其中包括一系列内部连接的层，并且每层内都包含一定数量的节点。每个节点都是一个神经元，对其输入应用某些操作并生成输出。输入层的节点由原始数据设置，而输出层则确定数据的类别。从输入层向前传递到输出层的过程称为模型推理。对于包含数十层、每层数百个节点的典型 DNN，参数数量很容易达到数百万的规模。因此，DNN 推理为 *computational intensive*。请注意，在本文中，我们重点关注 DNN 推理，因为 DNN 训练通常具有延迟容忍性，并且通常使用强大的云资源以离线方式进行。

B. Inefficiency of Device- or Edge-based DNN Inference

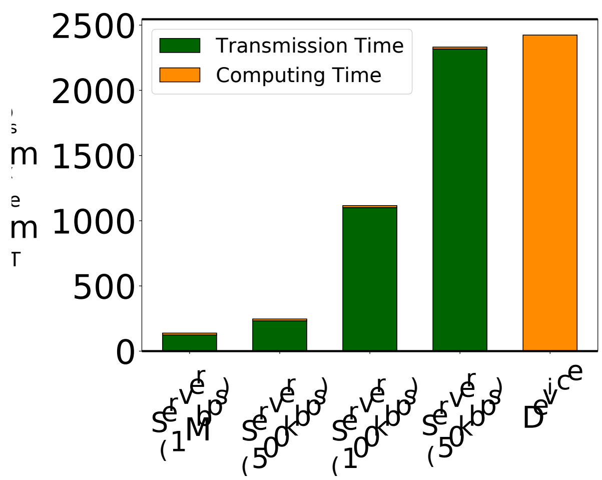

图 2. AlexNet 运行时

目前，移动DNN推理的现状是直接在移动设备上执行或卸载到云/边缘服务器上执行。不幸的是，这两种方法都可能面临性能较差（即端到端延迟）的问题，难以很好地满足实时智能移动应用（例如 AR/VR 移动游戏和智能机器人）[11]。作为说明，我们使用 Raspberry Pi 小型计算机和台式 PC 分别模拟移动设备和边缘服务器，运行经典的 AlexNet [12] DNN 模型，在 Cifar-10 数据集 [13] 上进行图像识别。图 2 绘制了边缘和移动设备之间不同带宽下不同方法的端到端延迟的细分。它清楚地表明，在资源有限的Raspberry Pi 上执行模型需要超过2s。此外，基于边缘的执行方法的性能主要取决于输入数据传输时间（边缘服务器计算时间保持在∼10ms），因此高度*sensitive*可用带宽。具体来说，随着可用带宽从 1Mbps 跃升至 50Kbps，端到端延迟从 0.123 秒攀升至 2.317 秒。考虑到实践中网络带宽资源的稀缺（例如，由于用户和应用程序之间的网络资源争用）以及移动设备上的计算资源限制，基于设备和基于边缘的方法都难以很好地支持许多具有严格延迟要求的新兴实时智能移动应用程序。

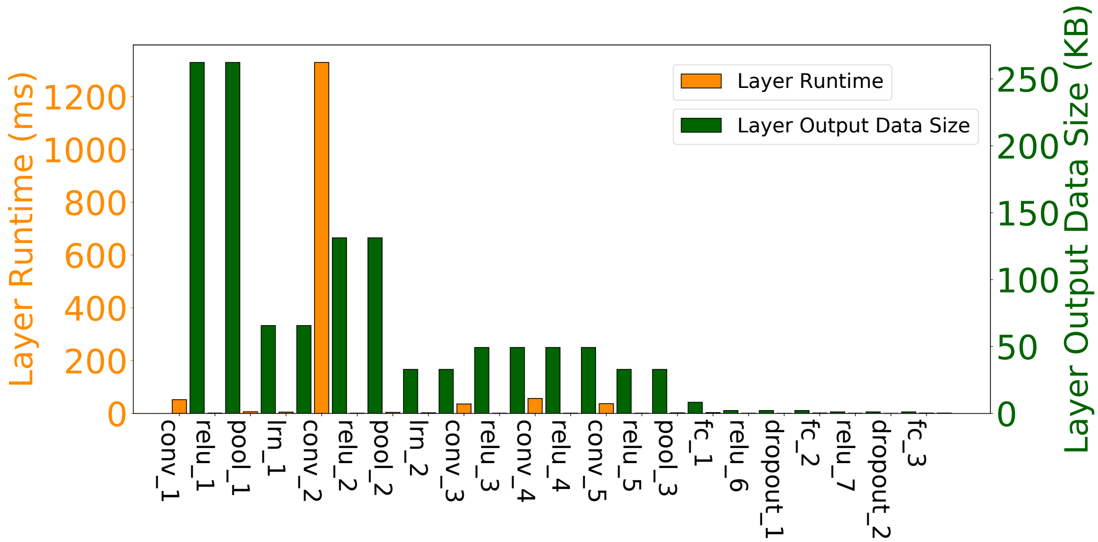

Figure 3. AlexNet Layer Runtime on Raspberry Pi

C. *Enabling Edge Intelligence with DNN Partitioning and Right-Sizing*

DNN Partitioning: For a better understanding of the performance bottleneck of DNN execution, we further break the runtime (on Raspberry Pi) and output data size of each layer in Fig. 3. Interestingly, we can see that the runtime and output data size of different layers exhibit great heterogeneities, and layers with a long runtime do not necessarily have a large output data size. Then, an intuitive idea is DNN partitioning, i.e., partitioning the DNN into two parts and offloading the computational intensive one to the server at a low transmission overhead, and thus to reduce the end-to-end latency. For illustration, we choose the second local response normalization layer (i.e., `lrn_2`) in Fig. 3 as the partition point and offload the layers before the partition point to the edge server while running the rest layers on the device. By DNN partitioning between device and edge, we are able to collaborate hybrid computation resources in proximity for low-latency DNN inference.

DNN Right-Sizing: While DNN partitioning greatly reduces the latency by bending the computing power of the edge server and mobile device, we should note that the optimal DNN partitioning is still constrained by the run time of layers running on the device. For further reduction of latency, the approach of *DNN Right-Sizing* can be combined with DNN partitioning. DNN right-sizing promises to accelerate model inference through an early-exit mechanism. That is, by training a DNN model with multiple exit points and each has a different size, we can choose a DNN with a small size tailored to the application demand, meanwhile to alleviate the computing burden at the model division, and thus to reduce the total latency. Fig. 4 illustrates a branchy AlexNet with five exit points. Currently, the early-exit mechanism has been supported by the open source framework BranchyNet[14]. Intuitively, DNN right-sizing further reduces the amount of computation required by the DNN inference tasks.

Problem Description: Obviously, DNN right-sizing incurs the problem of latency-accuracy tradeoff — while early-exit reduces the computing time and the device side, it also deteriorates the accuracy of the DNN inference. Considering the fact that some applications (e.g., VR/AR game) have stringent deadline requirement while can tolerate moderate accuracy loss, we hence strike a nice balance between the latency and the accuracy in an on-demand manner. Particularly, given the predefined and stringent latency goal, we maximize the accuracy without violating the deadline requirement. More specifically, the problem to be addressed in this paper can be summarized as: given a predefined latency requirement, how to jointly optimize the decisions of DNN partitioning and right-sizing, in order to maximize DNN inference accuracy.

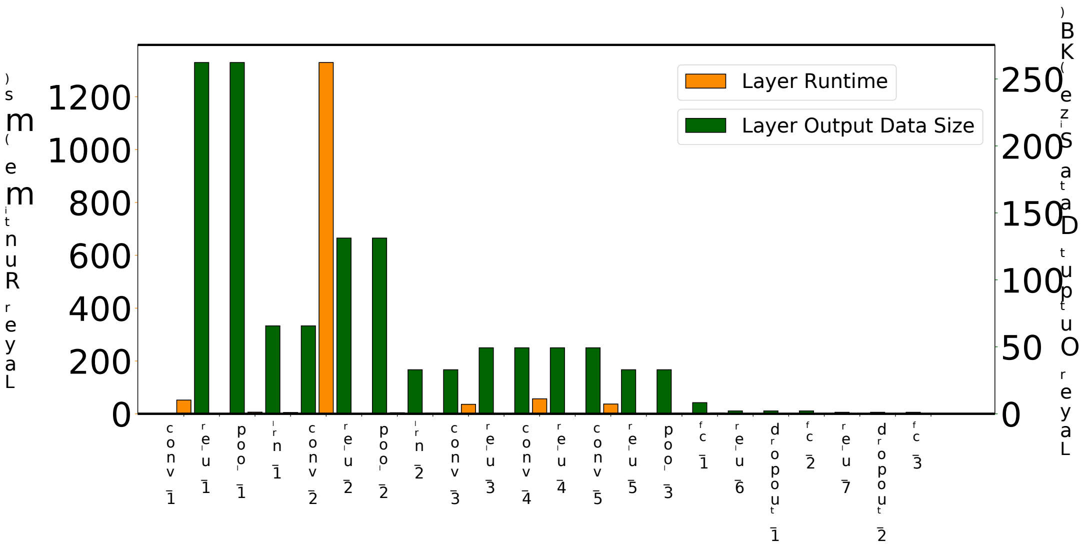

图 3. Raspberry Pi 上的 AlexNet 层运行时

C. Enabling Edge Intelligence with DNN Partitioning and Right-Sizing

DNN 分区：为了更好地理解 DNN 执行的性能瓶颈，我们进一步打破了图 3 中各层的运行时间（在 Raspberry Pi 上）和输出数据大小。有趣的是，我们可以看到不同层的运行时间和输出数据大小表现出很大的异质性，运行时间长的层不一定具有大的输出数据大小。然后，一种直观的想法是 DNN 分区，即将 DNN 分为两部分，并以较低的传输开销将计算密集型部分卸载到服务器，从而减少端到端延迟。为了便于说明，我们选择图 3 中的第二个本地响应归一化层（即 lrn_2）作为分区点，并将分区点之前的层卸载到边缘服务器，同时在设备上运行其余层。通过设备和边缘之间的 DNN 划分，我们能够协作邻近的混合计算资源以实现低延迟 DNN 推理。

DNN 调整大小：虽然 DNN 分区通过提高边缘服务器和移动设备的计算能力大大降低了延迟，但我们应该注意，最佳的 DNN 分区仍然受到设备上运行的层的运行时间的限制。为了进一步减少延迟，*DNN Right-Sizing* 的方法可以与 DNN 分区相结合。 DNN 规模调整有望通过提前退出机制加速模型推理。也就是说，通过训练具有多个出口点且每个出口点具有不同尺寸的DNN模型，我们可以根据应用需求选择较小尺寸的DNN，同时减轻模型划分时的计算负担，从而降低总延迟。图 4 展示了一个具有五个出口点的分支 AlexNet。目前，提前退出机制已经得到开源框架BranchyNet[14]的支持。直观上，DNN 规模调整进一步减少了 DNN 推理任务所需的计算量。

问题描述：显然，DNN 调整大小会带来延迟与精度权衡的问题——提前退出虽然减少了计算时间和设备端，但也降低了 DNN 推理的准确性。考虑到某些应用程序（例如 VR/AR 游戏）具有严格的截止时间要求，同时可以容忍适度的精度损失，因此我们以按需方式在延迟和精度之间取得了良好的平衡。特别是，考虑到预先定义的严格延迟目标，我们在不违反截止日期要求的情况下最大限度地提高了准确性。更具体地说，本文要解决的问题可以概括为：给定预定义的延迟要求，如何联合优化 DNN 分区和调整大小的决策，以最大化 DNN 推理精度。

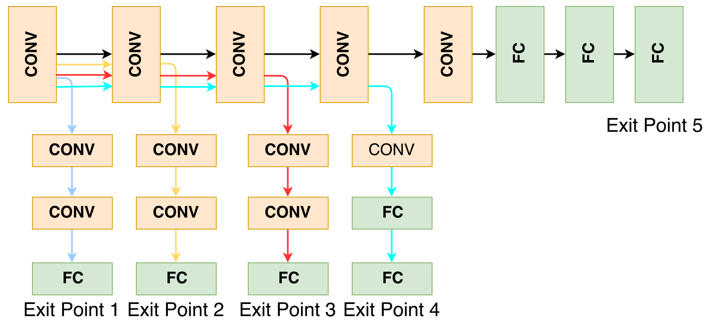

Figure 4. An illustration of the early-exit mechanism for DNN right-sizing

III. Framework

We now outline the initial design of Edgent, a framework that automatically and intelligently selects the best partition point and exit point of a DNN model to maximize the accuracy while satisfying the requirement on the execution latency.

A. System Overview

Fig. 5 shows the overview of Edgent. Edgent consists of three stages: offline training stage, online optimization stage and co-inference stage.

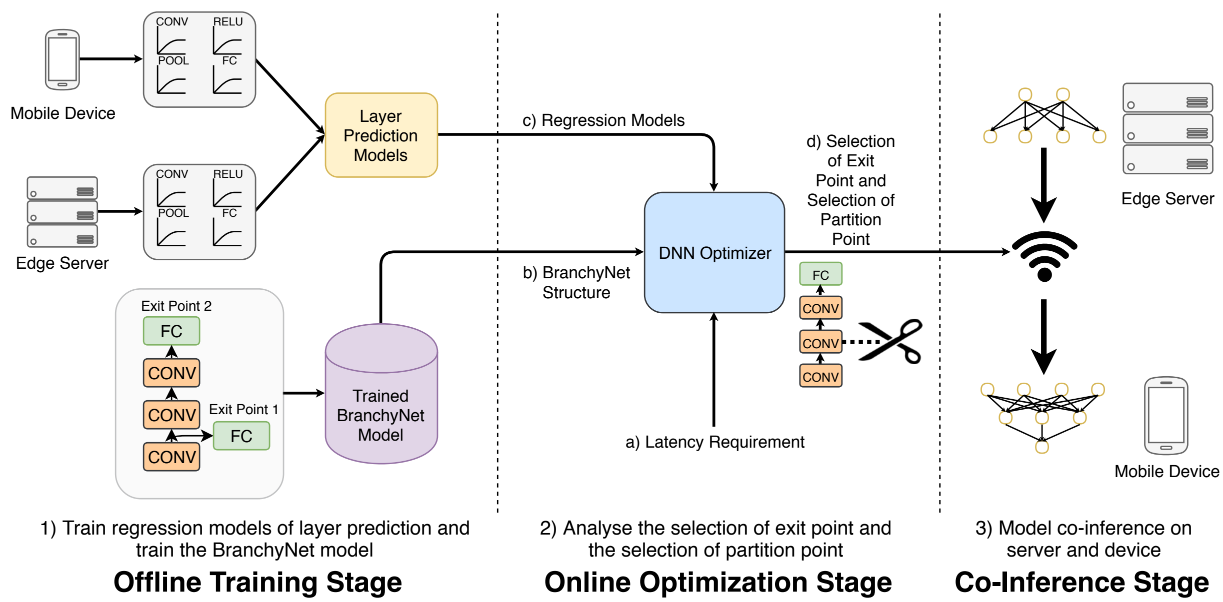

Figure 5. Edgent overview

At **offline training stage**, Edgent performs two initializations: (1) profiling the mobile device and the edge server to generate regression-based performance prediction models (Sec. III-B) for different types DNN layer (e.g., `Convolution`, `Pooling`, etc.). (2) Using `Branchynet` to train DNN models with various exit points, and thus to enable early-exit. Note that the performance profiling is infrastructure-dependent, while the DNN training is application-dependent. Thus, given the sets of infrastructures (i.e., mobile devices and edge servers) and applications, the two initializations only need to be done once in an offline manner.

At **online optimization stage**, the DNN optimizer selects the best partition point and early-exit point of DNNs to maximize the accuracy while providing performance guarantee on the end-to-end latency,

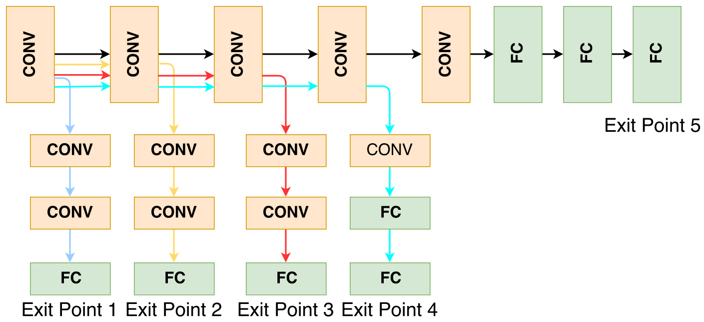

图 4. DNN 规模调整的提前退出机制图示

三． 框架

现在我们概述 Edgent 的初始设计，该框架可以自动、智能地选择 DNN 模型的最佳划分点和退出点，以最大限度地提高准确性，同时满足执行延迟的要求。

A. *System Overview*

Fig。图5显示了Edgent的概述。 Edgent由三个阶段组成：离线训练阶段、在线训练阶段奥蒂米化阶段和协同推理阶段。

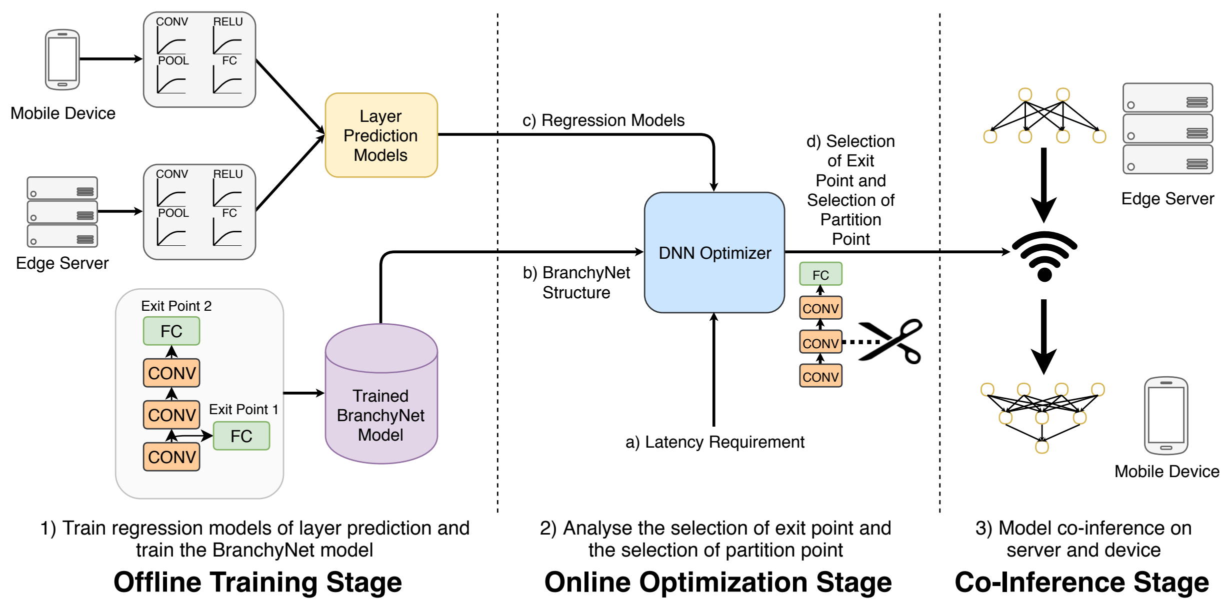

图 5. Edgent 概述

在离线训练阶段，Edgent 执行两个初始化：(1) 分析移动设备和边缘服务器，为不同类型的 DNN 层（例如，卷积、池化等）生成基于回归的性能预测模型（第 III-B 节）。 (2)使用Branchynet训练具有多种退出点的DNN模型，从而实现提前退出。请注意，性能分析依赖于基础设施，而 DNN 训练则依赖于应用程序。因此，给定一组基础设施（即移动设备和边缘服务器）和应用程序，这两个初始化只需要以离线方式完成一次。

在在线优化阶段，DNN优化器选择DNN的最佳划分点和早期退出点，以最大限度地提高准确性，同时为端到端延迟提供性能保证，

Table I
THE INDEPENDENT VARIABLES OF REGRESSION MODELS

Layer Type	Independent Variable
Convolution	amount of input feature maps, (filter size/stride)^2*(num of filters)
Relu	input data size
Pooling	input data size, output data size
Local Response Normalization	input data size
Dropout	input data size
Fully-Connected	input data size, output data size
Model Loading	model size

based on the input: (1) the profiled layer latency prediction models and Branchynet-trained DNN models with various sizes. (2) the observed available bandwidth between the mobile device and edge server. (3) The pre-defined latency requirement. The optimization algorithm is detailed in Sec. III-C.

At **co-inference stage**, according to the partition and early-exit plan, the edge server will execute the layer before the partition point and the rest will run on the mobile device.

B. Layer Latency Prediction

When estimating the runtime of a DNN, Edgent models the per-layer latency rather than modeling at the granularity of a whole DNN. This greatly reduces the profiling overhead since there are very limited classes of layers. By experiments, we observe that the latency of different layers is determined by various independent variables (e.g., input data size, output data size) which are summarized in Table I. Note that we also observe that the DNN model loading time also has an obvious impact on the overall runtime. Therefore, we further take the DNN model size as a input parameter to predict the model loading time. Based on the above inputs of each layer, we establish a regression model to predict the latency of each layer based on its profiles. The final regression models of some typical layers are shown in Table II (size is in bytes and latency is in ms).

Table II
REGRESSION MODEL OF EACH TYPE LAYER

Layer	Mobile Device model	Edge Server model
Convolution	y = 6.03e-5 $*$ x1 + 1.24e-4 $*$ x2 + 1.89e-1	y = 6.13e-3 $*$ x1 + 2.67e-2 $*$ x2 - 9.909
Relu	y = 5.6e-6 $*$ x + 5.69e-2	y = 1.5e-5 $*$ x + 4.88e-1
Pooling	y = 1.63e-5 $*$ x1 + 4.07e-6 $*$ x2 + 2.11e-1	y = 1.33e-4 $*$ x1 + 3.31e-5 $*$ x2 + 1.657
Local Response Normalization	y = 6.59e-5 $*$ x + 7.80e-2	y = 5.19e-4 $*$ x+ 5.89e-1
Dropout	y = 5.23e-6 $*$ x+ 4.64e-3	y = 2.34e-6 $*$ x+ 0.0525
Fully-Connected	y = 1.07e-4 $*$ x1 - 1.83e-4 $*$ x2 + 0.164	y = 9.18e-4 $*$ x1 + 3.99e-3 $*$ x2 + 1.169
Model Loading	y = 1.33e-6 $*$ x + 2.182	y = 4.49e-6 $*$ x + 842.136

C. Joint Optimization on DNN Partition and DNN Right-Sizing

At online optimization stage, the DNN optimizer receives the latency requirement from the mobile device, and then searches for the optimal exit point and partition point of the trained branchynet model. The whole process is given in Algorithm 1. For a branchy model with M exit points, we denote that the i-th exit point has N_i layers. Here a larger index i correspond to a more accurate inference model of larger size. We use the above-mentioned regression models to predict ED_j the runtime of the j-th layer when it runs on device and ES_j the runtime of the j-th layer when it runs on server. D_p is the output of the p-th layer. Under a specific bandwidth B, with the input data $Input$, then we calculate $A_{i,p}$ the whole runtime $(\sum_{j=1}^{p-1} ES_j + \sum_{k=p}^{N_i} ED_j + Input/B + D_{p-1}/B)$ when the p-th is the partition

表 1 回归模型自变量

Layer Type	Independent Variable
Convolution	amount of input feature maps, (filter size/stride)^2*(num of filters)
Relu	input data size
Pooling	input data size, output data size
Local Response Normalization	input data size
Dropout	input data size
Fully-Connected	input data size, output data size
Model Loading	model size

基于输入：(1) 不同大小的分析层延迟预测模型和 Branchynet 训练的 DNN 模型。 (2) 观察到的移动设备和边缘服务器之间的可用带宽。 (3) 预先定义的延迟要求。优化算法详见第 2 节。 III-C。

在协同推理阶段，根据分区和提前退出计划，边缘服务器将执行分区点之前的层，其余部分将在移动设备上运行。

B. Layer Latency Prediction

在估计 DNN 的运行时间时，Edgent 对每层延迟进行建模，而不是在整个 DNN 的粒度上进行建模。这大大减少了分析开销，因为层的类别非常有限。通过实验，我们观察到不同层的延迟是由各种自变量（例如输入数据大小、输出数据大小）决定的，这些变量总结在表 I 中。请注意，我们还观察到 DNN 模型加载时间也对整体运行时间有明显的影响。因此，我们进一步将DNN模型大小作为输入参数来预测模型加载时间。基于每层的上述输入，我们建立一个回归模型，根据其配置文件来预测每层的延迟。一些典型层的最终回归模型如表二所示（大小以字节为单位，延迟以毫秒为单位）。

表2 各类型层回归模型

Layer	Mobile Device model	Edge Server model
Convolution	y = 6.03e-5 ∗ x1 + 1.24e-4 ∗ x2 + 1.89e-1	y = 6.13e-3 ∗ x1 + 2.67e-2 ∗ x2 - 9.909
Relu	y = 5.6e-6 ∗ x + 5.69e-2	y = 1.5e-5 ∗ x + 4.88e-1
Pooling	y = 1.63e-5 ∗ x1 + 4.07e-6 ∗ x2 + 2.11e-1	y = 1.33e-4 ∗ x1 + 3.31e-5 ∗ x2 + 1.657
Local Response Normalization	y = 6.59e-5 ∗ x + 7.80e-2	y = 5.19e-4 ∗ x+ 5.89e-1
Dropout	y = 5.23e-6 ∗ x+ 4.64e-3	y = 2.34e-6 ∗ x+ 0.0525
Fully-Connected	y = 1.07e-4 ∗ x1 - 1.83e-4 ∗ x2 + 0.164	y = 9.18e-4 ∗ x1 + 3.99e-3 ∗ x2 + 1.169
Model Loading	y = 1.33e-6 ∗ x + 2.182	y = 4.49e-6 ∗ x + 842.136

C. Joint Optimization on DNN Partition and DNN Right-Sizing

在在线优化阶段，DNN 优化器接收来自移动设备的延迟要求，然后搜索训练后的 Branchynet 模型的最佳退出点和分区点。整个过程在算法 1 中给出。对于具有 M 出口点的分支模型，我们表示 i -th 出口点具有 N_i 层。这里较大的索引i对应于较大尺寸的更准确的推理模型。我们使用上述回归模型来预测 ED_j 在设备上运行时 j-th 层的运行时间，以及 ES_j 在服务器上运行时 j-th 层的运行时间。 D_p 是 p-th 层的输出。在特定带宽B下，输入数据$Input$，然后当p-th是分区时，我们计算$A_{i,p}$ 整个运行时间（$\sum_{j=1}^{p-1} ES_j + \sum_{k=p}^{N_i} ED_j + Input/B + D_{p-1}/B$）

point of the selected model of $i\text{-}th$ exit point.When $p = 1$, the model will only run on the device then $ES_p = 0, D_{p-1}/B = 0, Input/B = 0$ and when $p = N_i$, the model will only run on the server then $ED_p = 0, D_{p-1}/B = 0$. In this way, we can find out the best partition point having the smallest latency for the model of $i\text{-}th$ exit point. Since the model partition does not affect the inference accuracy, we can then sequentially try the DNN inference models with different exit points(i.e., with different accuracy), and find the one having the largest size and meanwhile satisfying the latency requirement. Note that since regression models for layer latency prediction are trained beforehand, Algorithm 1 mainly involves linear search operations and can be done very fast (no more than 1ms in our experiments).

Algorithm 1 Exit Point and Partition Point Search

Input:

M: number of exit points in a branchy model
$\{N_i | i = 1, \cdots, M\}$: number of layers in each exit point
$\{L_j | j = 1, \cdots, N_i\}$: layers in the $i\text{-}th$ exit point
$\{D_j | j = 1, \cdots, N_i\}$: each layer output data size of $i\text{-}th$ exit point
$f(L_j)$: regression models of layer runtime prediction
B: current wireless network uplink bandwidth
$Input$: the input data of the model
$latency$: the target latency of user requirement

Output:

Selection of exit point and its partition point

Procedure
for $i = M, \cdots, 1$ **do**
  Select the $i\text{-}th$ exit point
  for $j = 1, \cdots, N_i$ **do**
    $ES_j \leftarrow f_{edge}(L_j)$
    $ED_j \leftarrow f_{device}(L_j)$
  end for
  $A_{i,p} = \underset{p=1,\cdots,N_i}{\operatorname{argmin}} \left(\sum_{j=1}^{p-1} ES_j + \sum_{k=p}^{N_i} ED_j + Input/B + D_{p-1}/B\right)$
  if $A_{i,p} \leq latency$ **then**
    return Selection of Exit Point i and its Partition Point p
  end if
end for
return NULL ▷ can not meet latency requirement

IV. Evaluation

We now present our preliminary implementation and evaluation results.

A. Prototype

We have implemented a simple prototype of Edgent to verify the feasibility and efficacy of our idea. To this end, we take a desktop PC to emulate the edge server, which is equipped with a quad-core Intel processor at 3.4 GHz with 8 GB of RAM, and runs the Ubuntu system. We further use Raspberry Pi 3 tiny computer to act as a mobile device. The Raspberry Pi 3 has a quad-core ARM processor at 1.2 GHz with 1 GB of RAM. The available bandwidth between the edge server and the mobile device is controlled by the WonderShaper [15] tool. As for the deep learning framework, we choose Chainer [16] that can well support branchy DNN structures.

For the branchynet model, based on the standard AlexNet model, we train a branchy AlexNet for image recognition over the large-scale Cifar-10 dataset [13]. The branchy AlexNet has five exit points as showed in Fig. 4 (Sec. II), each exit point corresponds to a sub-model of the branchy AlexNet. Note that in Fig. 4, we only draw the convolution layers and the fully-connected layers but ignore other layers for ease

i-th出口点所选模型的点。当$p = 1$时，模型只会在设备上运行，然后$ES_p = 0, D_{p-1}/B = 0$, $Input/B = 0$，当$p = N_i$时，模型只会在服务器上运行，然后$ED_p = 0, D_{p-1}/B = 0$。这样，我们就可以找到i-th出口模型延迟最小的最佳划分点点。由于模型划分不会影响推理精度，因此我们可以依次尝试具有不同出口点（即具有不同精度）的 DNN 推理模型，找到尺寸最大且同时满足延迟要求的模型。请注意，由于层延迟预测的回归模型是预先训练的，因此算法 1 主要涉及线性搜索操作，并且可以非常快地完成（在我们的实验中不超过 1ms）。

算法1退出点和划分点搜索

输入：
M：分支模型中的出口点数量 $\{N_i|i = 1, \cdots, M\}$：每个出口点的层数 $\{L_j|j = 1, \cdots, N_i\}$：$i$-$th$出口点的层数 $\{D_j|j = 1, \cdots, N_i\}$：$i$-$th$出口点的每层输出数据大小 $f(L_j)$：层运行时预测的回归模型 B：当前无线网络上行带宽$Input$：模型的输入数据 $latency$：用户需求的目标延迟

输出：
退出点及其分割点的选择

1: 程序 2: for $i = M, \cdots, 1$ do 3: 选择 i-th 退出点 4: for $j = 1, \cdots, N_i$ do 5: $ES_j \leftarrow f_{edge}(L_j)$ 6: $ED_j \leftarrow f_{device}(L_j)$ 7: end for 8: $A_{i,p} = \text{argmin}_{p=1,\cdots,N_i} (\sum_{j=1}^{p-1} ES_j + \sum_{k=p}^{N_i} ED_j + Input/B + D_{p-1}/B)$ 9: if $A_{i,p} \leq latency$ then 10: 返回退出点 i 及其分区点的选择p 11: 结束 if 12: 结束 for 13: 返回 NULL

$\triangleright$ can not meet latency requirement

四. 评估

现在我们介绍一下我们的初步实施和评估结果。

A. Prototype

我们实现了一个简单的 Edgent 原型来验证我们想法的可行性和有效性。为此，我们使用一台台式机来模拟边缘服务器，该服务器配备了 3.4 GHz 四核 Intel 处理器，8 GB RAM，运行 Ubuntu 系统。我们进一步使用Raspberry Pi 3微型计算机作为移动设备。 Raspberry Pi 3 具有 1.2 GHz 四核 AR M 处理器和 1 GB RAM。边缘服务器和移动设备之间的可用带宽由 WonderShaper [15] 工具控制。至于深度学习框架，我们选择能够很好支持分支DNN结构的Chainer[16]。

对于branchynet模型，基于标准AlexNet模型，我们训练了branchy AlexNet，用于在大规模Cifar-10数据集上进行图像识别[13]。分支 AlexNet 有五个出口点，如图 4（第二节）所示，每个出口点对应于分支 AlexNet 的一个子模型。请注意，在图4中，为了方便起见，我们只绘制了卷积层和全连接层，而忽略了其他层

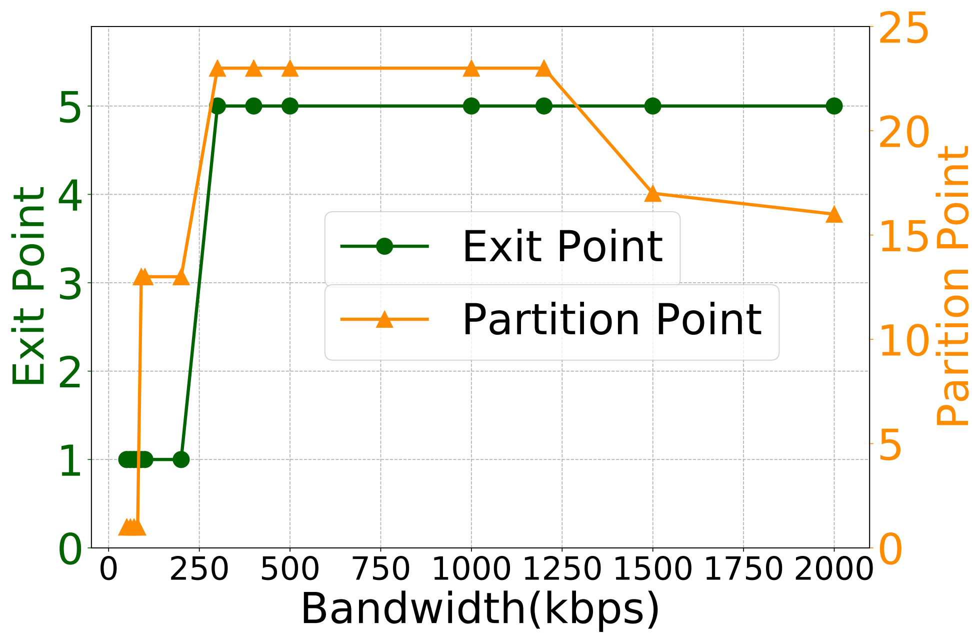

Figure 6. Selection under different bandwidths

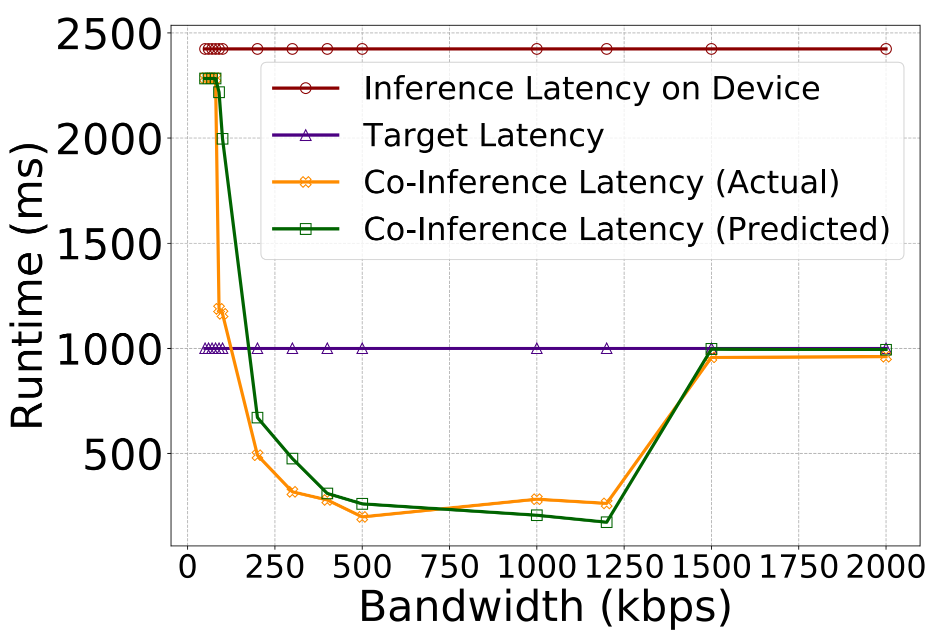

Figure 7. Model runtime under different bandwidths

of illustration. For the five sub-models, the number of layers they each have is 12, 16, 19, 20 and 22, respectively.

For the regression-based latency prediction models for each layer, the independent variables are shown in the Table. I, and the obtained regression models are shown in Table II.

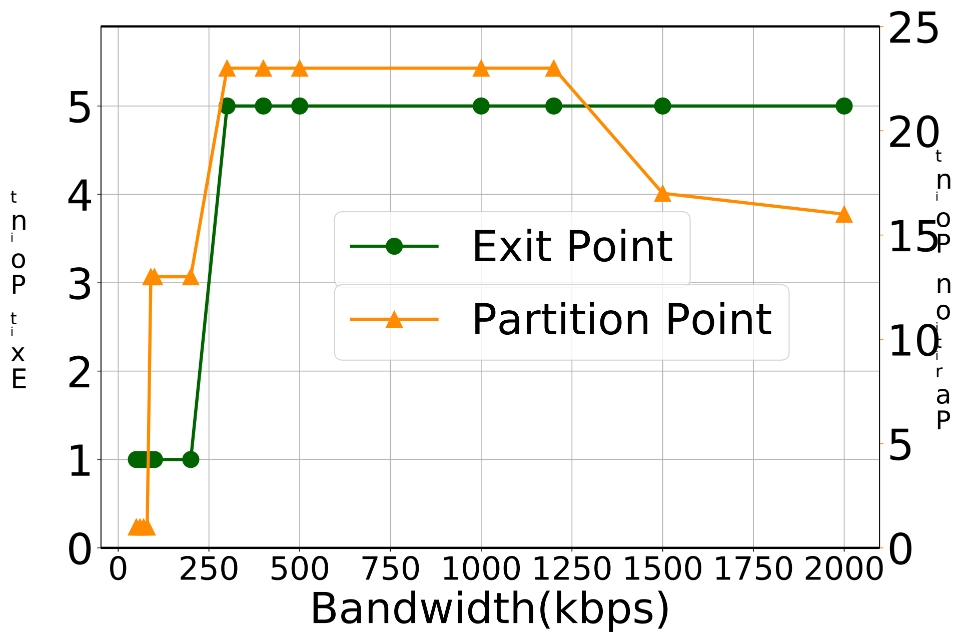

图6 不同带宽下的选择

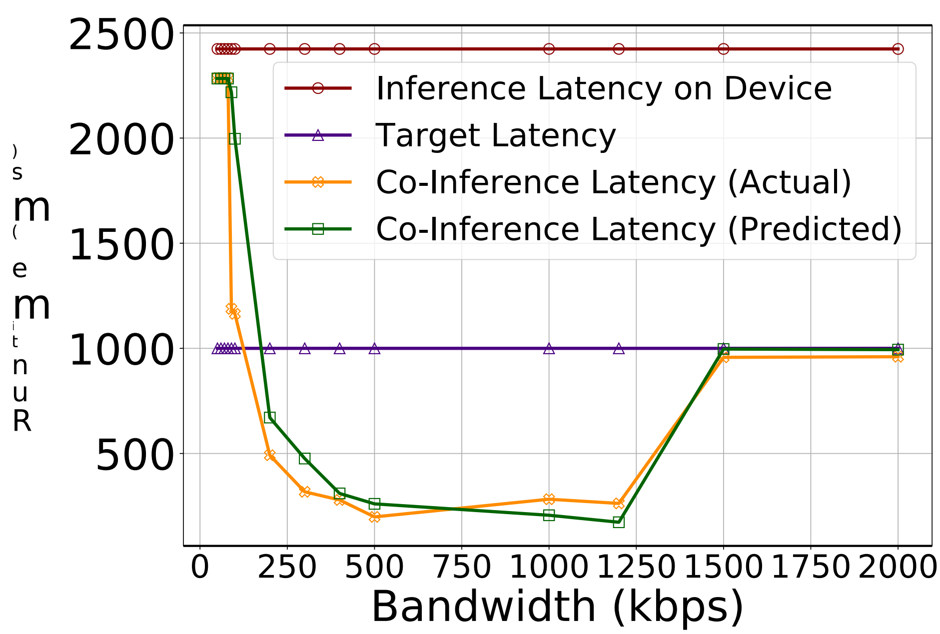

图 7. 不同带宽下的模型运行时间

的插图。对于五个子模型，它们的层数分别为 12、16、19、20 和 22。

对于每层基于回归的延迟预测模型，自变量如表所示。I，得到的回归模型如表II所示。

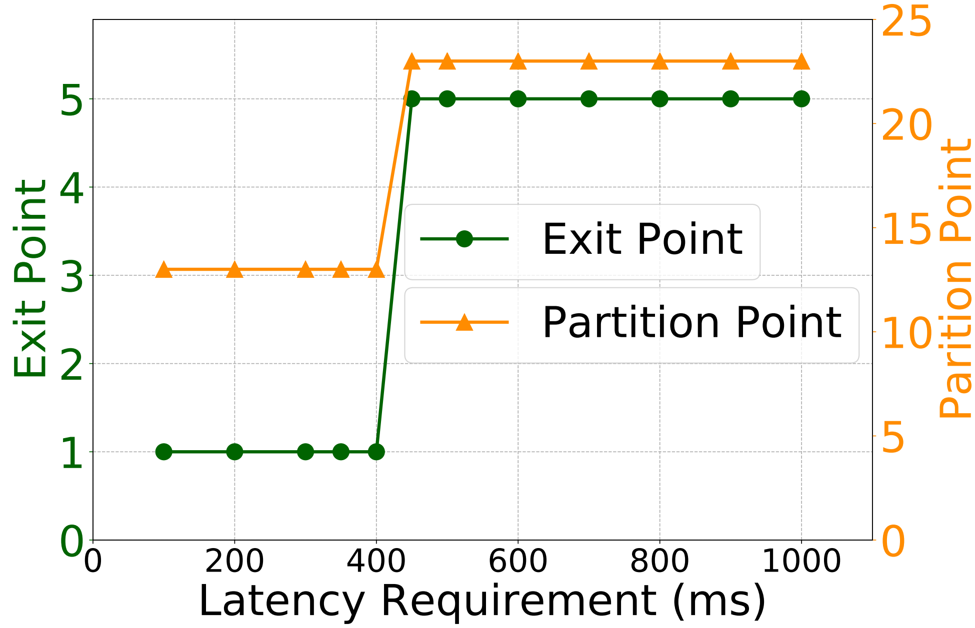

Figure 8. Selection under different latency requirements

B. Results

We deploy the branchynet model on the edge server and the mobile device to evaluate the performance of Edgent. Specifically, since both the pre-defined latency requirement and the available bandwidth play vital roles in Edgent's optimization logic, we evaluate the performance of Edgent under various latency requirements and available bandwidth.

We first investigate the effect of the bandwidth by fixing the latency requirement at 1000ms and varying the bandwidth 50kbps to 1.5Mbps. Fig. 6 shows the best partition point and exit point under different bandwidth. While the best partition points may fluctuate, we can see that the best exit point gets higher as the bandwidth increases. Meaning that the higher bandwidth leads to higher accuracy. Fig. 7 shows that as the bandwidth increases, the model runtime first drops substantially and then ascends suddenly. However, this is reasonable since the accuracy gets better while the latency is still within the latency requirement when increase the bandwidth from 1.2Mbps to 2Mbps. It also shows that our proposed regression-based latency approach can well estimate the actual DNN model runtime latency. We further fix the bandwidth at 500kbps and vary the latency from 100ms to 1000ms. Fig. 8 shows the best partition point and exit point under different latency requirements. As expected, the best exit point gets higher as the latency requirement increases, meaning that a larger latency goal gives more room for accuracy improvement.

In Fig. 9, under different latency requirements, it shows the model accuracy of different inference methods. The accuracy is negative if the inference can not satisfy the latency requirement. The network bandwidth is set to 400kbps. Seen in the Fig. 9, at a very low latency requirement (100ms), all four methods can't satisfy the requirement. As the latency requirement increases, inference by Edgent works earlier than the other methods that at the 200ms to 300ms requirements, by using a small model with a moderate inference accuracy to meet the requirements. The accuracy of the model selected by Edgent gets higher as the latency requirement relaxes.

V. Conclusion

In this work, we present Edgent, a collaborative and on-demand DNN co-inference framework with device-edge synergy. Towards low-latency edge intelligence, Edgent introduces two design knobs to

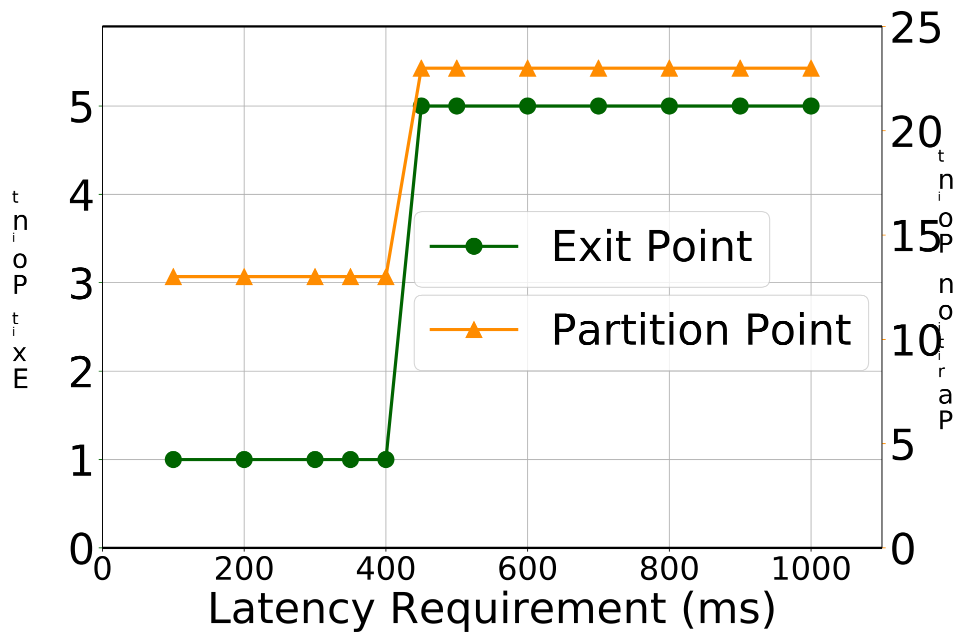

图8. 不同时延要求下的选择

B. Results

我们在边缘服务器和移动设备上部署branchynet模型来评估Edgent的性能。具体来说，由于预定义的延迟要求和可用带宽在 Edgent 的优化逻辑中都起着至关重要的作用，因此我们评估了 Edgent 在各种延迟要求和可用带宽下的性能。

我们首先通过将延迟要求固定为 1000 毫秒并将带宽从 50kbps 更改为 1.5Mbps 来研究带宽的影响。图6给出了不同带宽下的最佳划分点和退出点。虽然最佳划分点可能会波动，但我们可以看到，随着带宽的增加，最佳退出点会变得更高。这意味着更高的带宽会带来更高的精度。图 7 显示，随着带宽的增加，模型运行时间首先大幅下降，然后突然上升。然而，这是合理的，因为当带宽从 1.2Mbps 增加到 2Mbps 时，精度会提高，而延迟仍在延迟要求之内。它还表明我们提出的基于回归的延迟方法可以很好地估计实际的 DNN 模型运行时延迟。我们进一步将带宽固定为 500kbps，并将延迟从 100ms 更改为 1000ms。图8展示了不同延迟要求下的最佳划分点和退出点。正如预期的那样，随着延迟要求的增加，最佳退出点会变得更高，这意味着更大的延迟目标为准确性改进提供了更大的空间。

图9中，在不同延迟要求下，显示了不同推理方法的模型精度。如果推理不能满足延迟要求，则准确性为负。网络带宽设置为400kbps。从图9中可以看出，在非常低的延迟要求（100ms）下，四种方法都不能满足要求。随着延迟要求的增加，在 200ms 到 300ms 的要求下，Edgent 的推理比其他方法更早发挥作用，通过使用具有中等推理精度的小模型来满足要求。随着延迟要求的放宽，Edgent 选择的模型的准确性会变得更高。

五、结论

在这项工作中，我们提出了 Edgent，一种具有设备边缘协同作用的协作式按需 DNN 协同推理框架。为了实现低延迟边缘智能，Edgent 引入了两个设计旋钮

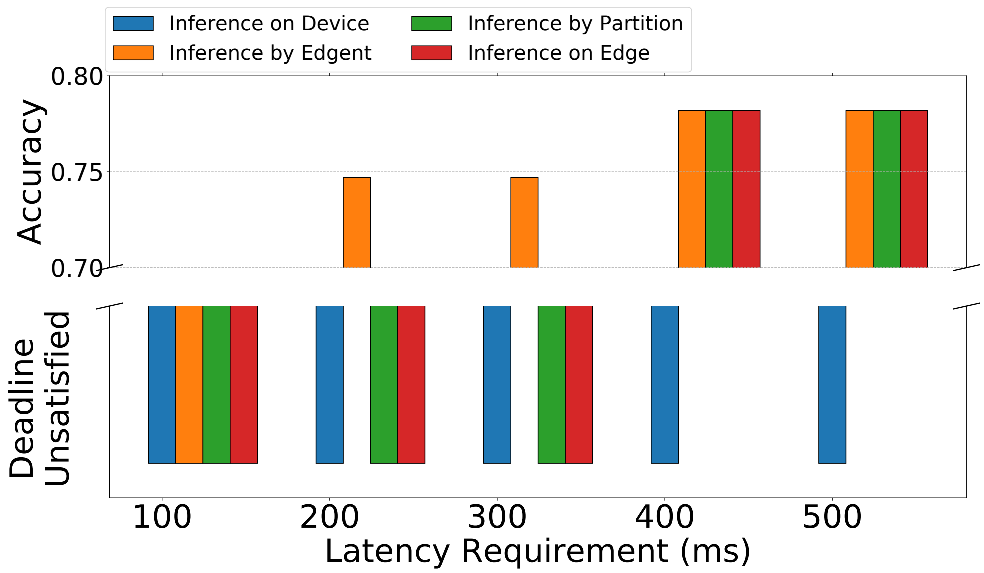

Figure 9. The accuracy comparison under different latency requirement

tune the latency of a DNN model: DNN partitioning which enables the collaboration between edge and mobile device, and DNN right-sizing which shapes the computation requirement of a DNN. Preliminary implementation and evaluations on Raspberry Pi demonstrate the effectiveness of Edgent in enabling low-latency edge intelligence. Going forward, we hope more discussion and effort can be stimulated in the community to fully accomplish the vision of intelligent edge service.

References

[1] C. e. a. Szegedy, "Going deeper with convolutions," 2014.

[2] A. V. D. e. a. Oord, "Wavenet: A generative model for raw audio," *arXiv:1609.03499*, 2016.

[3] D. Wang and E. Nyberg, "A long short-term memory model for answer sentence selection in question answering," in *ACL and IJCNLP*, 2015.

[4] Qualcomm, "Augmented and virtual reality: the first wave of 5g killer apps," 2017.

[5] S. Han, H. Mao, and W. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *Fiber*, 2015.

[6] Y. Kim, E. Park, S. Yoo, T. Choi, L. Yang, and D. Shin, "Compression of deep convolutional neural networks for fast and low power mobile applications," in *ICLR*, 2016.

[7] J. W. et al., "Quantized convolutional neural networks for mobile devices," 2016.

[8] N. Lane, S. Bhattacharya, A. Mathur, C. Forlivesi, and F. Kawsar, "Dxtk: Enabling resource-efficient deep learning on mobile and embedded devices with the deepx toolkit," 2016.

[9] F. N. I. et al., "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and $<$0.5mb model size," *arXiv:1602.07360*, 2016.

[10] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *2017 ASPLOS*, 2017.

[11] C. T. et al., "5g mobile/multi-access edge computing," 2017.

[12] K. Alex, I. Sutskever, and G. Hinton, "Imagenet classification with deep convolutional neural networks," in *NIPS*, 2012.

[13] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009.

[14] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd ICPR*, 2016.

[15] V. Mulhollon, "Wondershaper," http://manpages.ubuntu.com/manpages/trusty/man8/wondershaper.8.html, 2004.

[16] P. Networks, "Chainer," https://github.com/chainer/chainer/tree/v1, 2017.

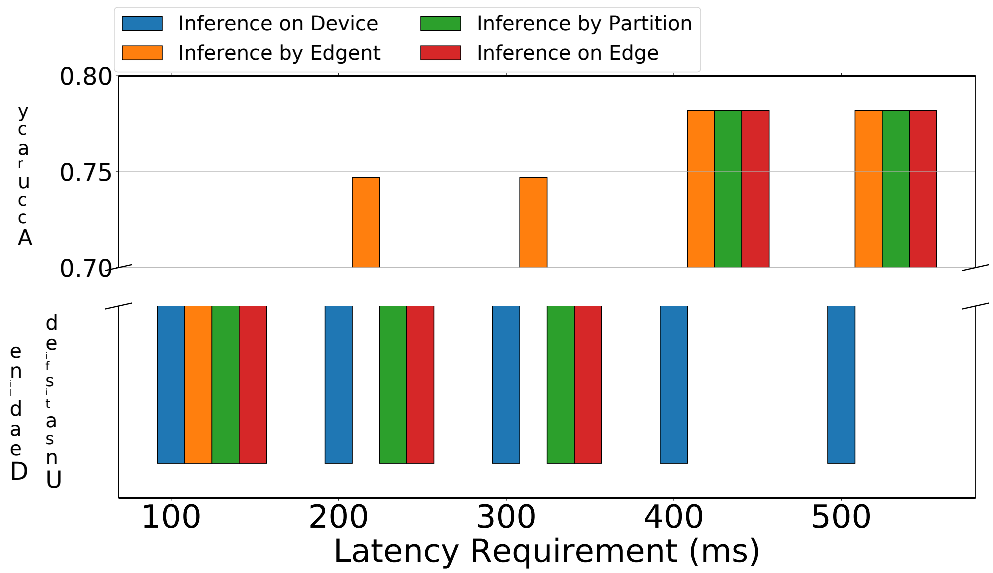

图9. 不同时延要求下的精度对比

调整 DNN 模型的延迟：DNN 分区可实现边缘设备和移动设备之间的协作，DNN 大小调整可满足 DNN 的计算要求。在 Raspberry Pi 上的初步实施和评估证明了 Edgent 在实现低延迟边缘智能方面的有效性。展望未来，我们希望能够激发社区更多的讨论和努力，以全面实现智能边缘服务的愿景。

参考

[1] C. e.一个。 Szegedy，“深入研究卷积”，2014 年。 [2] A. V. D. e.一个。 Oord，“Wavenet：原始音频生成模型”，*arXiv:1609.03499*，2016 年。 [3] D. Wang 和 E. Nyberg，“用于问答中答案句子选择的长短期记忆模型”，*ACL and IJCNLP*，2015 年。 [4] Qualcomm，“增强现实和虚拟现实：第一波 5G 杀手级应用程序”，2017 年。 [5] S. Han， H. Mao 和 W. Dally，“深度压缩：通过修剪、训练量化和霍夫曼编码来压缩深度神经网络”，*Fiber*，2015 年。 [6] Y. Kim、E. Park、S. Yoo、T. Choi、L. Yang 和 D. Shin，“用于快速和低功耗移动应用的深度卷积神经网络压缩”，*ICLR*，2016 年。 [7] J. W. et al.，“移动设备的量化卷积神经网络”，2016 年。 [8] N. Lane、S. Bhattacharya、A. Mathur、C. Forlivesi 和 F. Kawsar，“Dxtk：使用 deepx 工具包在移动和嵌入式设备上实现资源高效的深度学习”，2016 年。 [9] F. N. I. 等人，“Squeezenet：Alexnet 级精度参数减少 50 倍，模型大小为 <0.5mb”，*arXiv:1602.07360*，2016 年。[10] Y. Kang、J. Hauswald、C. Gau、A. Rovinski、T. Mudge、J. Mars 和 L. Tang，“神经外科医生：云端和移动边缘之间的协作智能”，*2017 ASPLOS*，2017 年。[11] C. T. 等人，“5g移动/多接入边缘计算，”2017 年。 [12] K. Alex、I. Sutskever 和 G. Hinton，“使用深度卷积神经网络进行 Imagenet 分类”，*NIPS*，2012 年。 [13] A. Krizhevsky，“从微小图像中学习多层特征”，2009 年。 [14] S. Teerapittayanon、B. McDanel 和 H. T. Kung，“Branchynet：通过早期退出深度神经网络进行快速推理”，*2016 23rd ICPR*，2016 年。[15] V. Mulhollon，“Wondershaper”，http://manpages.ubuntu.com/manpages/trusty/man8/wondershaper.8.html，2004 年。[16] P. Networks，“Chainer”，https://github.com/chainer/chainer/tree/v1，2017。